CHAPTER 3

Introduction to Coding

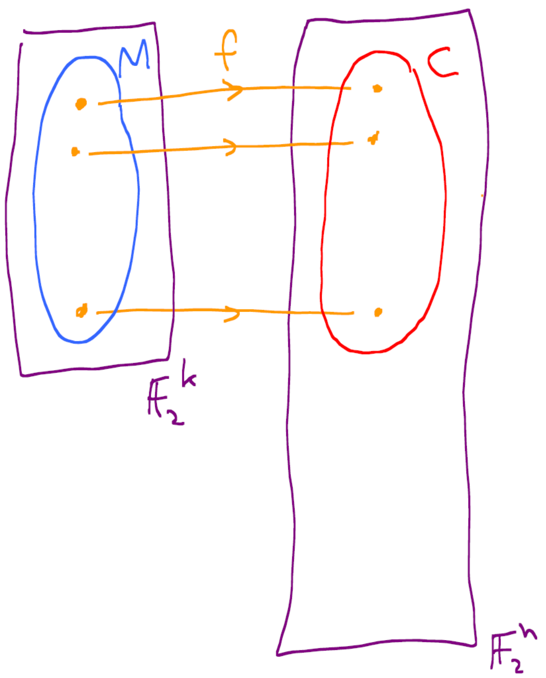
3.1. Basic Definitions & Setup

We give the definition of a binary block code, which we just call a code.

3.1.1 Definition Let $n \geq k$ be positive integers. A *(binary block) code* consists of:

- ★ A set $M \subseteq \mathbb{F}_2^k$ of *message words* together with
- ★ An injective function called the *encoding function* $f\colon M \to \mathbb{F}_2^n$.

3.1.1 Definition of a binary block (n,k)-code.



$f: M \longrightarrow C$ is a bijection

M: message words of length k

C: codewords of length $n \geqslant k$.

n: length of the code

k: message length
dimension of the code.

f^{-1} is the decoding function.

We write $C = f(M)$, the image of f, and (abusing terminology) call C the *code*. Elements in C are called *codewords*. The *length* of the code is n, and the *message length* or *dimension* of the code is k.

In fact we will always just take $M = \mathbb{F}_2^k$.

A code C is an (n, k)-code if it has codewords of length n and $|C| = 2^k$.

The encoding function viewed as a map $f\colon M \to C$ is a bijection, so $|M| = |C|$. The inverse function $f^{-1}\colon C \to M$ is the *decoding* function.

In general, if $f\colon M \to \mathbb{F}_2^n$ is a code with $f(M) = C$ we shall often refer only to C, and omit explicit mention of f when it can deduced. Thus we simply say that *a code C is a subset of $\mathbb{F}_2^n$*. However in the background there is always a function $f\colon M \to C$.

3.1.2 Definition Many (but not all) codes are obtained by taking message words of length k and (somehow) adding another $n-k$ *check bits* to each word, thus giving codewords of length n.

The encoding function in this case is

$$m = \overbrace{**\cdots*}^{k} \mapsto \overbrace{**\cdots*}^{k}\overbrace{\bullet\bullet\cdots\bullet}^{n-k}. \tag{3.1.1}$$

Note: not all codes work this way. In general the encoding process $M \to C$ can be any injective function, so it does not have to keep any of the bits of a message word fixed.

3.1.4 Definition An *m-fold repetition code* is formed by taking the 2^k message words of length k and forming the codewords of length km by writing down each word of length k a total of m times. (Compare Example 1.2.1).

Example: The 3-fold repetition code of dimension $k=2$.

$$\begin{aligned} 00 &\xrightarrow{f} 000000 \\ 01 &\longrightarrow 010101 \\ 10 &\longrightarrow 101010 \\ 11 &\longrightarrow 111111 \end{aligned} \qquad n=6.$$

3.1.5 Example Let $M = \mathbb{F}_2^3$, and let $f\colon \mathbb{F}_2^3 \to \mathbb{F}_2^4$ be defined by

$$f(x_1, x_2, x_3) = (x_1, x_2, x_3, x_1 + x_2 + x_3).$$

This is similar[1] to the raid5 example. The last bit is called a *(parity) check bit*.

Explicitly: $f(000) = 0000$, $f(001) = 0011$, $f(010) = 0101$, $f(011) = 0110$, $f(100) = 1001$, $f(101) = 1010$, $f(110) = 1100$, $f(111) = 1111$. So $C = \{0000, 0011, 0101, 0110, 1001, 1010, 1100, 1111\}$.

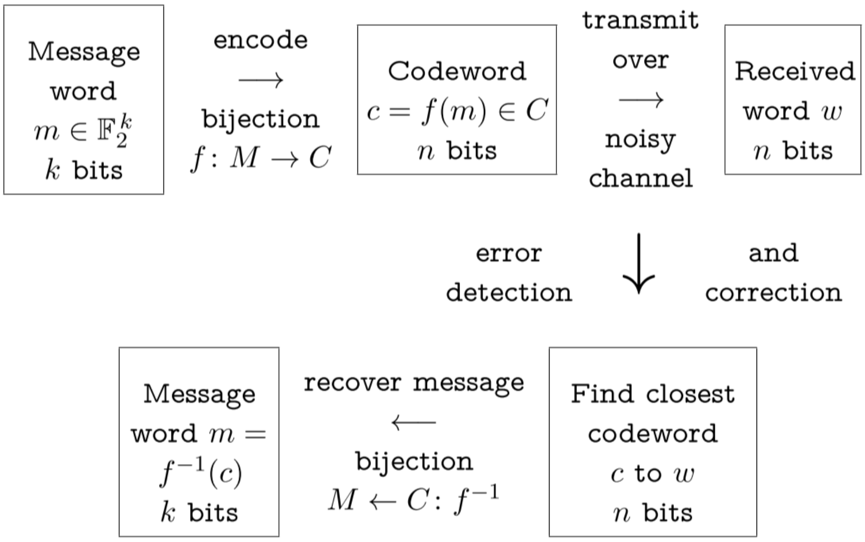
Exercise: Convince yourself that in this code, every word has even weight. If a single bit of a codeword is changed, the result will be an odd weight word, and we will detect an error.

Encoding & Decoding Process

Consider an (n, k)–code, $f : \mathbb{F}_2^k \to C$.

Encode each message word before transmission. Only codewords will ever be transmitted. Therefore, if any non-codeword is received, error(s) must have occurred.

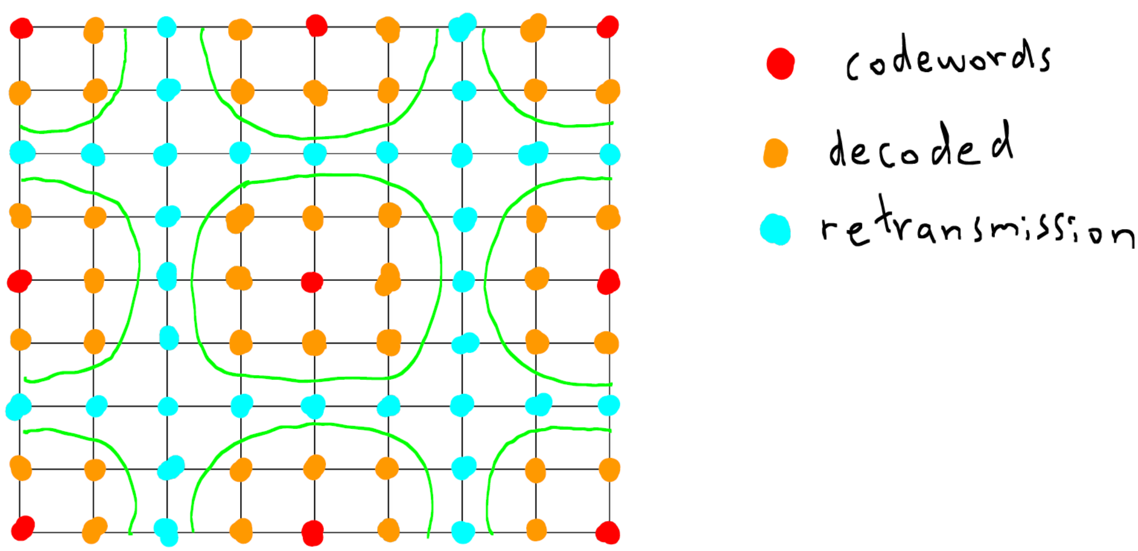
- ★ Suppose we receive a word w (with n bits).
- ★ If w is a codeword ($w \in C$), assume no errors occurred. Recover the message word $m = f^{-1}(w)$.
- ★ If w is not a codeword ($w \notin C$) one or more errors must have occurred. If there is a unique nearest codeword c to w, assume c was the word transmitted. Recover the message word $m = f^{-1}(c)$.



"Nearest codeword" means at smallest Hamming distance. At the last step, if there are two or more equally close codewords, we cannot decode successfully. Formally:

3.1.6 Definition *Incomplete Maximum Likelihood Decoding* or *IMLD*. Let C be the set of codewords, and w the word received. If there is a unique codeword $c \in C$ such that $d(w, c) < d(w, b)$ for any other $b \in C$, $b \neq c$, then we decode w as c. If there are two or more equally close codewords to w, we request retransmission (=decoding fails).

Unless stated otherwise, we will always assume that IMLD is being used.



Recall that only codewords are ever sent. Suppose a word is received that is not a codeword. Then we have *detected* that at least one error has occurred in transmission.

3.2.1 Definition Suppose that the codeword $c \in \mathbb{F}_2^n$ was transmitted, and the word w was received. The corresponding *error pattern* or *error* is defined to be the word e for which $c + e = w$.

You can think of e as the vector that gets added to codeword c to produce the received word w.

$\|e\|$ is the number of errors that occured.

Note that for binary codes $w = c + e \Leftrightarrow e = c + w$.

3.2.2 Definition A code C *detects* an error pattern $e \neq \mathbf{0} \iff c + e$ is not a codeword, for every $c \in C$. If $c + e \in C$ for some $c \in C$ then we say C cannot detect error pattern e (meaning: at least sometimes, cannot detect.)

Let $e \neq \mathbf{0}$ be an error pattern.

★ If $c + e \in C$ ever occurs (for any $c \in C$) then C cannot detect e.

★ To show that C can detect e, we must show that $c + e \notin C$ for every $c \in C$.

3.2.5 Definition A code C is said to be *t-error detecting* if it detects all non-zero error patterns of weight $\leq t$.

Exercise Show that the 3-fold repetition code of dimension 2 is 2-error detecting.

3.2.6 Definition The *distance* (or *minimum distance*) δ of a code is the smallest distance between any pair of distinct codewords:

$$\delta = \min_{b \neq c} d(b,c) = \min_{b \neq c} \|b+c\| \qquad \forall\, b \neq c \in C.$$

An (n,k)–code with minimum distance δ will be described as an (n,k,δ)*–code.*

3.2.7 Theorem Let C be a code with minimum distance δ. Then C will detect all non-zero error patterns of weight $< \delta$. There is at least one error pattern of weight δ that C will not detect. That is, C is $(\delta-1)$-error detecting, but not δ-error detecting.

Note that a code C with minimum distance δ may possibly detect *some* error patterns of weight δ or more, but it does not detect *all* error patterns of weight δ.

3.2.8 Lemma A code C <u>cannot</u> detect an error pattern $e \neq 0$ iff $e = c_1 + c_2$ for some distinct $c_1, c_2 \in C$:

The set of undetectable non-zero error patterns is $(C+C) \setminus \{0\}$.

Suppose we detect that errors have occurred. We would like to be able to correct them.

Suppose c was sent, error e occurred, so we received $w = c + e$. In the decoding process we find the closest codeword c^* to w, and assume that c^* was actually sent. This succeeds provided $c^* = c$. That is, provided c is the closest codeword to $w = c + e$. This breaks down once so many error occurs that $c + e$ is equally close, or strictly closer, to some other codeword c' than to c.

3.3.1 Definition A code C *corrects an error pattern* $e \neq \mathbf{0}$ if, for all $c \in C$, $c + e$ is strictly closer to c than to any other word $c' \in C$:

$$\forall c,\ c' \in C \qquad c' \neq c \implies d(c+e,c) < d(c+e,c').$$

3.3.2 Definition A code C is said to be t-*error correcting* if it corrects all non-zero error patterns of weight $\leq t$.

Let $e \neq \mathbf{0}$ be an error pattern.

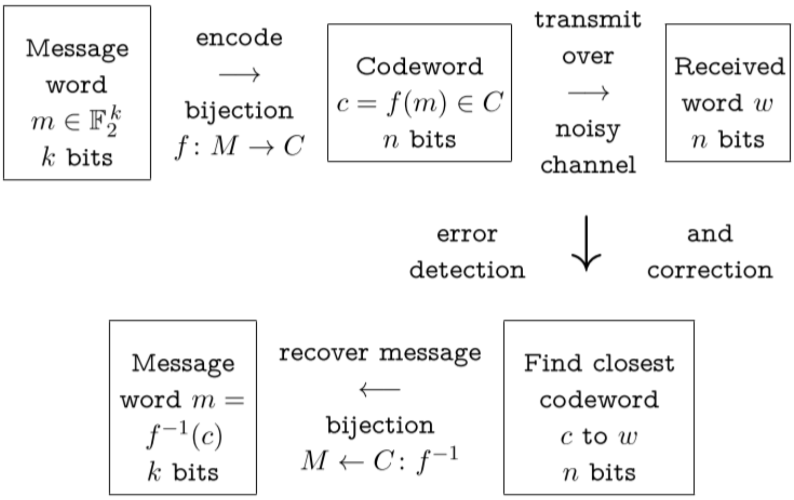
★ If for any $c,\ c' \in C$ with $c \neq c'$ we have $d(c+e,c) \geq d(c+e,c')$ then C cannot correct e.

★ To show that C can correct e, we must show that $d(c+e,c) < d(c+e,c')$ for every $c \neq c' \in C$.

3.3.4 Theorem Let C be a code with minimum distance δ. Then C can correct all non-zero error patterns with weight $< \delta/2$. This is best possible: there is at least one error pattern of weight $\lceil \delta/2 \rceil$ that C cannot correct.

3.3.5 Corollary Let C be a code with minimum distance δ. Then C is $\left\lfloor \dfrac{\delta - 1}{2} \right\rfloor$-error correcting.

Summary



k - length of message words (dimension) $\quad M = \mathbb{F}_2^k$

n - length of codewords $\quad C \subseteq \mathbb{F}_2^n \quad |C| = 2^k$

δ - min. distance between any 2 codewords

An (n, k, δ)-code is $(\delta - 1)$-error detecting.

An (n, k, δ)-code is $\left\lfloor \frac{\delta - 1}{2} \right\rfloor$-error correcting.

<u>Exercise</u> For each of the following codes, determine the parameters (n, k, δ) and state the maximum values of s and t for which the code is s-error detecting and t-error correcting.

(a) The 2-fold repetition code of dimension 3.

(b) The x-fold repetition code of dimension 2, where $x \in \mathbb{Z}^+$.

(c) $C = \{0000, 1000, 0111, 1111\}$

(d) The parity-check code of dimension 3. (For each word in $\mathbb{F}_2^3$, add a bit so that the resulting codeword has even weight.)

Consider the 2-fold repetition code of dimension 3 and the parity check code of dimension 3. (Parts (a) and (d) of the previous exercise.)

These codes have the same dimension and the same distance.

Which is more efficient?

3.4.1 Definition If C is an (n,k) code, its *rate* is $r = k/n$.

Note that $0 \le r \le 1$ (and in practice $0 < r < 1$).

In the case where the set M of message words may not be all of $\mathbb{F}_2^k$ a more general definition of rate is

$$\frac{1}{n}\log_2 |C|.$$

A code with $\delta = 3$ is quite impressive when $n = 7$, but not so impressive if n is large. Consider the *relative distance* $\partial = \delta/n$. This approximately represents the fraction of bits that may contain errors and still be detected, while $\partial/2$ is approximately the fraction of bits that can be corrected.

Because of Theorems 3.2.7 and 3.3.4, a main goal of coding theory is to construct codes with large minimum distance. For small values of n and k the maximal possible value of δ for which an (n,k,δ) code exists is known, but for general n and k this is an open problem. We will consider bounds on δ in terms of n and k later. Ideally we would like both r and ∂ close to 1, but we shall see that this is impossible.

In practice, one must balance the decrease in rate against the increase in accuracy obtained by using a code. Factors that influence the choice of code for a particular application include the reliability of the channel, the possibility of re-transmitting data if required and the sensitivity of the data to uncorrected errors.

3.5. Probability

For most of the course, we shall assume that errors are distributed uniformly, independently, with a fixed probability. (We shall consider other situations at the end of the course.) In detail:

Let $0 \leq p \leq 1$. We assume our binary channel has the following properties:

★ Errors in distinct positions are independent.

★ If we send a 0 through our binary channel, we receive a 0 with probability p. If we send 1, we receive 1 also with probability p.

A channel with these properties is called a *binary symmetric channel* or *BSC* of *reliability* p.

So reliability $p = 0.9$ means a 10% chance that any given bit is corrupted. Note:

★ If the channel has $p \leq 0.5$, it can be converted to a channel with $0.5 \leq p \leq 1$, by adding $\mathbf{1}$ to each received word. So we may assume $p \geq 0.5$.

★ If $p = 0.5$ the channel is perfectly random. Every possible received word of length n is equally probable, no matter what message was sent. There is no correlation between what is sent and what is received. This renders error detecting and correcting impossible to achieve. We assume this is not the case, so $p > 0.5$.

★ If $p = 1$ there are never errors and there is no need for a code, so assume $p < 1$.

from now on: assume we have a BSC with reliability p with $0.5 < p < 1$.

3.5.1 Definition Let $\mathbb{P}_p(c \to w)$ be the probability that if c is transmitted then w is received.

Exercise: Determine a formula for $\mathbb{P}_p(c \to w)$ in terms of n, p, c, w.

<u>Exercise</u>: Suppose that 10^6 bits per second are transmitted over a BSC with reliability $p = 0.999999$. What is the expected frequency of words with undetected errors if:

(a) The trivial code of all words in $\mathbb{F}_2^8$ is used?

(b) The parity check code of dimension 8 is used, that is, the code consisting of all even weight words in $\mathbb{F}_2^9$?

3.5.4 Theorem Suppose communication is via a BSC with reliability p with $0.5 < p < 1$. Let c_1 and c_2 be codewords of length n, and let w be any word of length n. Then

$$\mathbb{P}_p(c_1 \to w) \ \geq\ \mathbb{P}_p(c_2 \to w) \qquad \Longleftrightarrow \qquad d(c_1,\, w) \leq d(c_2,\, w).$$

In other words, the closer a codeword is to w, the more likely it is that it was the word sent.

Proof First note that $0.5 < p < 1 \implies 0 < 1-p < p$. Hence $\dfrac{p}{1-p} > 1$. Let $d_1 = d(w,\, c_1)$, $d_2 = d(w,\, c_2)$. Then:

$$\begin{array}{rcclr}
 & \mathbb{P}_p(c_1 \to w) & \geq & \mathbb{P}_p(c_2 \to w) & \\
 & \| & & \| & \\
\Longleftrightarrow & p^{n-d_1}(1-p)^{d_1} & \geq & p^{n-d_2}(1-p)^{d_2} & \text{by Equation (3.5.2)} \\
\Longleftrightarrow & \left(\dfrac{1-p}{p}\right)^{d_1} & \geq & \left(\dfrac{1-p}{p}\right)^{d_2} & \\
\Longleftrightarrow & \left(\dfrac{1-p}{p}\right)^{d_1-d_2} & \geq & 1 & \\
 & \| & & & \\
\Longleftrightarrow & \left(\dfrac{p}{1-p}\right)^{d_2-d_1} & \geq & 1 & \\
\Longleftrightarrow & d_2 & \geq & d_1 & \text{because } \dfrac{p}{1-p} > 1.
\end{array}$$

$\square$

Since $d(c_1, w) = \|c_1 + w\|$, we can restate Theorem 3.5.4 as

$$\mathbb{P}_p(c_1 \to w) \geq \mathbb{P}_p(c_2 \to w) \iff \|c_1 + w\| \leq \|c_2 + w\|.$$

The most likely codeword sent is the one with the error pattern of smallest weight.